Inlämningsuppgift 2, Funktionell Programmering: JAVA25, JAVAD25

Inlämningsuppgift 2, del 1 (av 2): För G och VG

Registrera er och titta igenom siten Advent of Code, finns på: https://adventofcode.com/

Kolla igenom hur uppgifterna är uppbyggda och notera att det finns adventskalendrar från 2015 och framåt (skriv året sist i url:en för att hitta de gamla kalendrarna). Notera hur det finns ett litet test-dataset på varje uppgifts sida, och sen en länk till ett större ”ordinarie” dataset. Notera även att varje uppgift har en a- och en b-del. B-delen visas först när man löst a-delen.

I denna inlämningsuppgift ska ni **välja** ut **minst 3** (men gärna fler) uppgifter från AoC att jobba med.

EN av uppgifterna ska du lösa**, i Kotlin**, utifrån vad vi tidigare har lärt oss, **utan hjälp från AI eller att titta på andra lösningar**. Detta är för att säkerställa att ni har lärt er och behärskar grundläggande Kotlin. Det finns inga krav på att din lösning måste vara snygg eller bra, men den måste funka (dvs. lösa problemet)

De andra två uppgifterna (välj gärna några av de svårare uppgifterna här) kan du lösa precis hur du vill. Börja med att tänka ut hur uppgifterna borde kunna lösas och börja jobba på en lösning själv, så långt du kan. Gå sedan ut på nätet och kolla runt hur andra programmerare har löst uppgiften. Titta på vilka olika datastrukturer och algoritmer andra utvecklare har valt att använda sig av. Fundera över hur lösningarna skiljer sig åt och vad olika design-val får för konsekvenser. Om du stöter på kod som du inte förstår, kolla upp hur den funkar och sätt dig in i vad som händer.

Bra källor för att hitta lösningar online:

- **Reddit** har sidor för varje dag av AoC, där folk postar sina lösningar. Notera att Reddit bara visar 200 inlägg i taget, i de aktuella trådarna finns över 1000 inlägg och ibland måste man, **manuellt**, scrolla ner en bra bit för att hitta Kotlin-lösningar. När ni tittar på kod som finns i andra människors gitHub-repon, tänk på att den kan ha delat upp koden mellan olika klasser och paket. Ibland kan det vara oklart vad som är flashiga Kotlin-funktioner och vad som är folks egna, hemmaskrivna, grejer. Men man märker om man kollar av importerna och rotar runt i filbiblioteken.
- **Todd Ginsberg** har en fantastisk adventskalender där han pedagogiskt förklarar hur han har löst varje uppgift: https://todd.ginsberg.com/post/advent-of-code/2025/ (ändra url:en för att se andra år):
- Låt **AI** förseslå olika sätt att lösa uppgiften och förklara kod du inte förstår.

Krav för G (och VG):

- Lös **minst 3** AoC-uppgifter enligt instruktionerna ovan.
- För uppgifter publicerade dag 1-5 **måste BÅDE** a- och b-uppgifterna göras. För uppgifter för dag 6 och högre är det ok att bara göra a-uppgiften.
- Lösningen måste funka med det **stora** data-setet, som ges på AoC-hemsidan.
- Kommentera så att det tydligt framgår vilken uppgift du löst (år och dag i december)
- Alla uppgifterna löses i **Kotlin**
- **Det är obligatoriskt att till 100% förstå varje rad kod du lämnar in/redovisar.**

Redovisning av denna uppgift ska vara ett **kunskapsutbyte** med resten av klassen. När du redovisar:

- Välj ut **EN** av de AoC-uppgifter du gjort, förslagsvis den uppgift du känner att du lärde dig mest av eller där du har hittat den intressantaste lösningen.
- Presentera din uppgift för klassen så alla förstår vad den går ut på.
- Berätta om några olika sätt att lösa uppgiften och berätta vilka för- och nackdelar de olika metoderna har.
- Lyft fram **minst en** intressant Kotlin-funktion, feature eller datatyp, som vi inte pratat om på kursen, men som hjälper till att lösa din AoC-uppgift. Presentera den för gruppen så att alla förstår hur den funkar och varför den är lämplig att använda i lösningen av din uppgift.

Tips på hur du väljer lämpliga AoC-uppgifter och läser in data:

- De första årens AoC-uppgifter har inte så många lösningar postade online. Därför kan det vara svårt att hitta tillräckligt många, intressanta, lösningar.
- 2023-25 års AoC var svårare än föregående år (gissningsvis för att folk inte bara ska kunna fråga AI om lösningar rakt av), **så år 2018-2022 kan vara bäst år att börja titta på.**
- Uppgifterna tenderar att bil successivt svårare ju längre in i december man kommer, men ökningen i svårighet är inte linjär. Ibland dyker det upp en jättesvår uppgift tidigt och ibland kommer en lättare uppgift sent. Om du märker att den uppgift du försökt dig på är jättesvår, så kan det vara bra att bara lägga uppgiften åt sidan och ta en annan uppgift så länge.
- Ni väljer själva vilken svårighetsgrad ni lägger er på. Det är, tekniskt sett, ok att välja 3 uppgifter från olika Dag 1, men ju högre ni lägger ribban för er själva, ju mer kommer ni att lära er.
- Ett enkelt sätt att läsa in datat till ditt program är genom följande:
    - Kopiera datat till en fil i filsystemet. För att läsa in, gör:
    - ```
      val data: List<String> =
      File("src/AoC/AoC2023/Dec5/input.txt").readLines()
      ```
    - Byt ut pathen ovan mot det som stämmer för dig.
    - Du kommer ni att ha en lista där varje element motsvarar en rad i ursprungsdatat

När du jobbar, passa på att ta lite tid till att njuta av excentrismen och mångfalden i Reddit-trådarna. Det finns så otroligt många programmeringsspråk, så många olika programmeringsstilar och så många olika sätt att lösa uppgifterna. Alla som, under redovisningen, kan berätta om ”en helt sjuk programmeringsgrej man upptäckte på Reddit” får en extra osynlig guldstjärna.

Denna uppgift löses individuellt.
      ```

Inlämningsuppgift, del 2 – Rekursion , enbart för VG, görs individuellt,

Tomtarna på Nordpolen har en strikt chefs-hierarki (rita gärna upp denna på papper):
Högsta chefen för allt är "Tomten"
Under "Tomten" jobbar "Glader" och "Butter"
Under "Glader" jobbar "Tröger", "Trötter" och "Blyger"
Under "Butter" jobbar "Rådjuret", "Nyckelpigan", "Haren" och "Räven"
Under "Trötter" jobbar "Skumtomten"
Under "Skumtomten" jobbar "Dammråttan"
Under "Räven" jobbar "Gråsuggan" och "Myran"
Under "Myran" jobbar "Bladlusen"

Er uppgift har 2 (ev. 3) delar:

- Koda tomtens chefshierarki i en lämplig datastruktur (som du själv väljer och definierar). Det finns flera tänkbara val här, ett krav är att varje nisse bara får kopplas mot nissar som är direkt över- och underordnade i er datastruktur.
    - Exempel på datastrukturer kan vara en Map med tomtenamn som nyckel och en lista av direkt underordnade som values, **eller** listor av par, bestående av överordnads/underordnads namn.
    - Direkta mappningar över flera nivåer är inte tillåtna. (Ett exempel på otillåten direkt mappning vore att ha Räven som nyckel i en Map och som värde lägga till en lista med Gråsuggan, Myran och Bladlusen. Här är det Bladlusen som är problemet, eftersom hen inte jobbar direkt under Räven, de andra är ok)
- Skriv en **rekursiv** funktion där man anger en nisses namn, som inparameter, och får tillbaka en lista på alla dess underordnade, på samtliga nivåer.
    - Exempel: Om man skriver in ”Räven” får man tillbaka en lista med Gråsuggan, Myran och Bladlusen.
    - **Inga tomtenamn får vara hårdkodade i den rekursiva loopen** (det är ok att hårdkoda namn i skapandet av datastrukturen).
- Om ni inte använder Sigruns kodskelett (se nedan), skriv egna tester som testar att **samtliga** tomtar får rätt underlydande när vi läser ut dem mha. ert program

Koden ska skrivas i Kotlin.

Det finns ett kodskelett, med färdiga tester bland Sigruns repo-filer. Det är ok att använda sig av dessa. Om du kan lösa uppgiften, men inte med de kodsignaturer Sigrun använt så är det ok att skriva egna, **OM** du också bifogar enhetstester som passar ihop med din kod så att det snabbt går att se att programmet funkar. Kodskelett och tester kan ni hitta på:

https://github.com/sigrunolafsdottir/JavaFuncProg/blob/master/src/Inl%C3%A4mningsuppgift2/Tomteland.kt

https://github.com/sigrunolafsdottir/JavaFuncProg/blob/master/test/Inl%C3%A4mningsuppgift2/TomtelandTest.java

Redovisning görs genom att ni visar er kod för Sigrun och **kör enhetstesterna** så vi ser att allt går grönt och funkar som det ska. På plats-klassen redovisar torsdagen den 12 mars, distans-klassen kör fredagen den 13 mars. Även VG-uppgiften görs enskilt. Lycka till!!!